



Politechnika Łódzka

Instytut Automatyki

Zakład sterowania robotów

TWORZENIE SYSTEMÓW ROBOTYCZNYCH

Temat 5

ROS2 Parameters | Launch files

10 marca 2025



Spis treści

1	Wprowadzenie	2
1.1	Przykłady zastosowania parametrów	2
1.2	Pliki <code>.launch</code>	2
2	Pliki konfiguracyjne	2
2.1	Wcięcia zamiast tabulatorów	3
2.2	Unikalne klucze	3
2.3	Typy danych	3
2.4	Listy i słowniki	3
2.5	Aliasowanie i referencje	3
2.6	Komentarze w YAML	4
2.7	Styl blokowy i wierszowy	4
2.8	Walidacja YAML	4
3	Komendy w terminalu	4
3.1	Wyświetlenie listy parametrów	4
3.2	Odczyt wartości parametru	4
3.3	Zmiana/ustawienie wartości parametru	4
3.4	Wyświetlenie wartości wszystkich parametrów	4
3.5	Wczytanie wartości parametrów z pliku	5
3.6	Wczytanie pliku konfiguracyjnego przy uruchomieniu węzła	5
4	Obsługa parametrów w symulacji turtlesim - terminal	5
4.1	Lista parametrów	5
4.2	Pobranie wartości parametrów dla koloru tła	6
4.3	Zmiana koloru tła na czerwony	6
4.4	Wyświetlenie wartości wszystkich parametrów dla węzła	7
4.5	Zapis wartości parametrów dla węzła do pliku <code>.yaml</code>	7
4.6	Wczytanie wartości parametrów dla węzła z pliku <code>.yaml</code>	8
4.7	Uruchomienie węzła razem z plikiem konfiguracyjnym <code>.yaml</code>	8
5	Procedura utworzenia węzła obsługującego parametry	8
5.1	Przejsięcie do lokalizacji pakietu <code>example_tsr_project</code>	8
5.2	Utworzenie pliku <code>parameters_node.py</code>	9
5.3	Napisanie kodu dla węzła	9
5.4	Aktualizacja <i><code>setup.py</code></i>	10
5.5	Modyfikacja <code>package.xml</code>	11
5.6	Budowanie paczki	11
5.7	Weryfikacja działania	11

6	Procedura dodania nowych plików konfiguracyjnych do pakietu	12
6.1	Przejsć do pakietu <code>example_tsr_project</code>	13
6.2	Utworzenie podkatalogu <code>config</code>	13
6.3	Utworzenie plików konfiguracyjnych <code>.yaml</code>	13
6.4	Aktualizacja <i>setup.py</i>	13
6.5	Budowanie pakietu	14
6.6	Weryfikacja działania	14
7	Procedura dodania nowego pliku <code>.launch</code>	14
7.1	Przejsć do pakietu <code>example_tsr_project</code>	14
7.2	Utworzenie podkatalogu <code>launch</code>	15
7.3	Utworzenie plików <code>.launch</code>	15
7.4	Aktualizacja <i>setup.py</i>	16
7.5	Budowanie pakietu	16
7.6	Weryfikacja działania	17
7.6.1	Weryfikacja działania pliku <code>topic_example_launch.py</code>	17
7.6.2	Weryfikacja działania pliku <code>multiple_nodes_launch.py</code>	17
8	Dokumentacja i inne linki	18

1 Wprowadzenie

Parametry w ROS2 umożliwiają konfigurację węzłów (nodes) bez konieczności modyfikacji kodu źródłowego. Dzięki nim można dostosować zachowanie aplikacji w czasie uruchomienia lub dynamicznie w trakcie jej działania. Czas życia parametrów związany jest z czasem życia węzła.

Każdy węzeł w ROS2 może mieć zestaw parametrów, które są organizowane w klucz-wartość. Obsługiwane typy wartości to:

```
1 bool
2 int64
3 float64
4 string
5 byte[]
6 bool[]
7 int64[]
8 float64[]
9 string[]
```

Każdy węzeł wykorzystujący parametry powinien mieć na początku zadeklarowaną ich listę. Chociaż istnieje możliwość uruchomienia węzła bez zadeklarowania parametrów.

Domyślnie nie można zmienić typu parametru w trakcie działania, gdyż kończy się to błędem. Istnieje możliwość zmiany konfiguracji parametru tak, co pozwoli na zastosowanie dynamicznej zmiany jego typu.

1.1 Przykłady zastosowania parametrów

- Ustawienie minimalnej i maksymalnej wartości prędkości postępowej i obrotowej robota,
- Definiowanie progów detekcji czujników
- Konfiguracja czujników
- Próg naładowania baterii poniżej, którego robot wraca do ładowarki.
- Tolerancja dojazdu do punktu

1.2 Pliki .launch

Pliki `launch` w ROS2 służą do uruchamiania wielu węzłów jednocześnie oraz konfiguracji ich parametrów, tematów i zależności. Umożliwiają automatyczne ładowanie plików konfiguracyjnych, definiowanie kolejności uruchamiania komponentów oraz dostosowywanie środowiska pracy, co jest szczególnie przydatne w bardziej złożonych systemach robotycznych.

2 Pliki konfiguracyjne

Pliki wykorzystywane do przechowywania informacji o konfiguracji parametrów zapisane są z użyciem języka YAML (YAML Ain't Markup Language). Jest to format umożliwiający przechowywanie danych i ich reprezentację w ustrukturyzowanej formie.

Najważniejsze zasady przy tworzeniu pliku `.yaml`.

2.1 Wcięcia zamiast tabulatorów

Pliki YAML używają spacji(zalecane 2 lub 4) do oznaczania poziomów hierarchii – nie wolno stosować tabulatorów.

```
1 robot:
2   model: "TurtleBot3"
3   max_speed: 1.2
```

2.2 Unikalne klucze

Każdy klucz w danej sekcji musi być unikalny.

Poprawne:

```
1 sensor:
2   - "Lidar"
3   - "Camera"
```

Niepoprawne:

```
1 sensor: "Lidar"
2 sensor: "Camera"
```

2.3 Typy danych

YAML obsługuje różne typy danych, takie jak liczby, ciągi znaków i wartości logiczne.

```
1 liczba: 42
2 nazwa: "Robot"
3 status: true
```

2.4 Listy i słowniki

Przykład listy:

```
1 sensory:
2   - lidar
3   - kamera
4   - sonar
```

Przykład słownika:

```
1 robot:
2   nazwa: "TurtleBot3"
3   v_max: 1.5
4   sensory:
5     lidar: true
6     kamera: false
```

2.5 Aliasowanie i referencje

```
1 domyślne_ustawienia: &default
2   max_speed: 1.5
3   min_speed: 0.1
4
5 robot_1:
6   <<: *default
7   nazwa: "TurtleBot3"
8
9 robot_2:
10  <<: *default
11  nazwa: "Create3"
```

2.6 Komentarze w YAML

```

1 # Konfiguracja robota
2 robot:
3   model: "TurtleBot3" # Typ modelu
4   max_speed: 1.2      # Maksymalna prędkość

```

2.7 Styl blokowy i wierszowy

Styl blokowy (zalecany):

```

1 robot:
2   model: "TurtleBot3"
3   sensory:
4     lidar: true
5     kamera: false

```

Styl wierszowy:

```

1 robot: {model: "TurtleBot3", sensory: {lidar: true, kamera: false}}

```

2.8 Walidacja YAML

Pliki YAML można sprawdzać narzędziami takimi jak `yamllint`.

```

1 yamllint config.yaml

```

3 Komendy w terminalu

3.1 Wyświetlenie listy parametrów

```

1 ros2 param list

```

3.2 Odczyt wartości parametru

```

1 ros2 param get <node_name> <parameter_name>

```

W miejscach `<>` należy wstawić odpowiednio nazwę węzła i nazwę parametru.

3.3 Zmiana/ustawienie wartości parametru

```

1 ros2 param set <node_name> <parameter_name> <value>

```

W miejscach `<>` należy wstawić odpowiednio nazwę węzła, nazwę i wartość parametru.

3.4 Wyświetlenie wartości wszystkich parametrów

```

1 ros2 param dump <node_name>

```

W miejscu <> należy wpisać nazwę węzła.

3.5 Wczytanie wartości parametrów z pliku

```
1 ros2 param load <node_name> <parameter_file>
```

W miejscach <> należy wstawić odpowiednio nazwę węzła oraz ścieżkę do pliku konfiguracyjnego.

3.6 Wczytanie pliku konfiguracyjnego przy uruchomieniu węzła

```
1 ros2 run <package_name> <executable_name> --ros-args --params-file <file_name>
```

W miejscach <> należy wstawić odpowiednio nazwę paczki, nazwę pliku wykonywalnego (nazwa węzła lub plik launch) a na końcu w <> wstawiana jest ścieżka do pliku konfiguracyjnego.

4 Obsługa parametrów w symulacji turtlesim - terminal

Zastosowanie komend w terminalu zostanie zaprezentowane na przykładzie symulacji turtlesim. Do tego wymagane jest uruchomienie w oddzielnych terminalach symulacji oraz sterowania:

```
1 ros2 run turtlesim turtlesim_node
2 ros2 run turtlesim turtle_teleop_key
```

4.1 Lista parametrów

Po uruchomieniu symulacji w nowym terminalu możemy sprawdzić listę dostępnych parametrów:

```
1 ros2 param list
```

Listę wszystkich parametrów przedstawia (Rys. 4.1)

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param list
/teleop_turtle:
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  scale_angular
  scale_linear
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
```

Rys. 4.1: Lista parametrów

Pod przestrzenią nazw `/turtlesim` znajdują się parametry pochodzące od węzła od symulacji robota. Natomiast przestrzeń nazw `/teleop_turtle` związana jest z uruchomionym węzłem od sterowania robotem z klawiatury.

Unikalne parametry dla symulacji `turtlesim` związane są z ustawieniem koloru tła (`background_b`, `background_g`, `background_r`). Wartości mogą być ustawiane z zakresu 0 – 255.

Natomiast dla węzła od sterowania ręcznego ustawiane są współczynniki skalujące prędkość postępową i obrotową odpowiednio `scale_linear` i `scale_angular`.

4.2 Pobranie wartości parametrów dla koloru tła

```
1 ros2 param get /turtlesim background_b
2 ros2 param get /turtlesim background_g
3 ros2 param get /turtlesim background_r
```

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param get /turtlesim background_b
Integer value is: 255
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param get /turtlesim background_g
Integer value is: 86
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param get /turtlesim background_r
Integer value is: 69
```

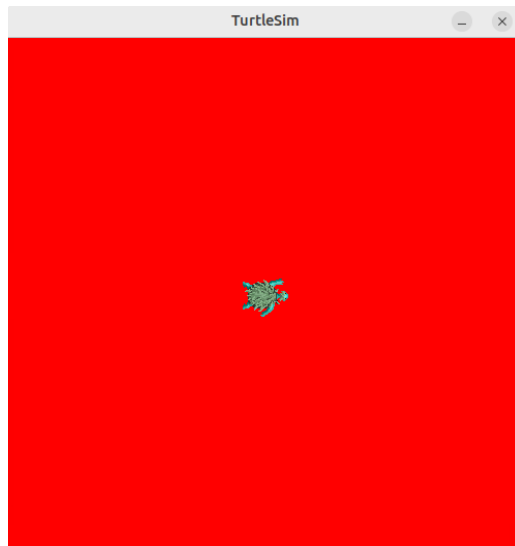
Rys. 4.2: Odczyt wartości parametrów dla koloru tła

4.3 Zmiana koloru tła na czerwony

```
1 ros2 param set /turtlesim background_b 0
2 ros2 param set /turtlesim background_g 0
3 ros2 param set /turtlesim background_r 255
```

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param set /turtlesim background_b 0
Set parameter successful
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param set /turtlesim background_g 0
Set parameter successful
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param set /turtlesim background_r 255
Set parameter successful
```

Rys. 4.3: Ustawienie nowych wartości parametrów dla koloru tła



Rys. 4.4: Symulacja turtlesim z czerwonym tłem.

Zmiana koloru jest aktywna podczas bieżącej sesji. Wyłączenie węzła od symulacji i ponowne jej uruchomienie spowoduje przywrócenie domyślnego koloru tła.

4.4 Wyświetlenie wartości wszystkich parametrów dla węzła

Dla węzła od symulacji `turtlesim` wyświetlona zostanie lista wszystkich parametrów wraz z ich wartościami:

```
1  ros2 param dump /turtlesim
```

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param dump turtlesim
/turtlesim:
  ros__parameters:
    background_b: 0
    background_g: 0
    background_r: 255
    qos_overrides:
      /parameter_events:
        publisher:
          depth: 1000
          durability: volatile
          history: keep_last
          reliability: reliable
    use_sim_time: false
```

Rys. 4.5: Wartości wszystkich parametrów dla węzła `turtlesim`

4.5 Zapis wartości parametrów dla węzła do pliku `.yaml`

Jest to polecenie podobne do poprzedniego. Różnica jest w pojawiającym się przekierowaniu, które powoduje zapisanie wyniku, który pojawiłby się w terminalu do pliku. Jeśli podana zostanie sama nazwa pliku to będzie on zapisany w lokalizacji, z której uruchomione było polecenie.

```
1  ros2 param dump /turtlesim > turtlesim.yaml
```

W wyniku działania polecenia parametry zostały zapisane do pliku `turtlesim.yaml`.

4.6 Wczytanie wartości parametrów dla węzła z pliku .yaml

W tej samej lokalizacji należy uruchomić polecenie do wczytania pliku. Niektóre parametry nie mogą zostać ustawione, gdyż są parametrami tylko do odczytu a ich wartości uwzględniane są tylko podczas inicjalizacji węzła.

```
1 ros2 param load /turtlesim turtlesim.yaml
```

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param load /turtlesim turtlesim.yaml
Set parameter background_b successful
Set parameter background_g successful
Set parameter background_r successful
Set parameter qos_overrides./parameter_events.publisher.depth failed: parameter 'qos_overrides./parameter_events.publisher.depth' cannot be set because it is read-only
Set parameter qos_overrides./parameter_events.publisher.durability failed: parameter 'qos_overrides./parameter_events.publisher.durability' cannot be set because it is read-only
Set parameter qos_overrides./parameter_events.publisher.history failed: parameter 'qos_overrides./parameter_events.publisher.history' cannot be set because it is read-only
Set parameter qos_overrides./parameter_events.publisher.reliability failed: parameter 'qos_overrides./parameter_events.publisher.reliability' cannot be set because it is read-only
Set parameter use_sim_time successful
```

Rys. 4.6: Wartości wszystkich parametrów dla węzła turtlesim

4.7 Uruchomienie węzła razem z plikiem konfiguracyjnym .yaml

Podejście umożliwia dodatkowo wczytanie nowych wartości parametrów, które są tylko do odczytu. Rozwiązuje to problem z wczytaniem nowych wartości parametrów jaki pojawiał się po użyciu `ros2 param load`.

```
1 ros2 run turtlesim turtlesim_node --ros-args --params-file turtlesim.yaml
```

5 Procedura utworzenia węzła obsługującego parametry

Zostanie utworzony węzeł do obsługi parametrów. Będzie on deklarował parametry, które mogą być dynamicznie zmieniane lub są tylko do odczytu. Węzeł będzie odczytywał wartości parametrów. Ponadto przedstawiona zostanie dynamiczna aktualizacja parametrów oraz sposób użycia odczytanych parametrów przez węzeł.

Węzeł wykorzystuje symulację `turtlesim`, w której wysyła maksymalną ustawioną wartość prędkości kątowej. Z tego powodu należy zapewnić, że sterowanie z klawiatury jest wyłączone oraz każdy inny węzeł, który wysyła prędkości na temacie `/turtle1/cmd_vel`.

5.1 Przejście do lokalizacji pakietu `example_tsr_project`

Przejdź do pakietu, w którym ma zostać utworzony nowy węzeł do obsługi parametrów (pakiet z plikami projektowymi `example_tsr_project`).

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

- `~` - oznacza lokalizację katalogu domowego
- `example_tsr_ws` - utworzony workspace. W katalogu `src` znajdują się wszystkie pakiety (nowe lub pobrane np. z githuba).

5.2 Utworzenie pliku parameters_node.py

W pakiecie `example_tsr_project` należy utworzyć nowy plik `parameters_node.py` w katalogu `example_tsr_project` (w kolejnym katalogu o tej samej nazwie a nie w głównym pakiecie gdzie znajdują się pliki `package.xml` i `setup.py`) a następnie nadać uprawnienia do wykonania. Brak uprawnień jest jednym z występujących problemów z widocznością węzła co uniemożliwia jego uruchomienie pomimo poprawnej konfiguracji paczki.

Należy wpisać kolejne polecenia w terminalu (należy znajdować się w głównej lokalizacji pakietu `example_tsr_project`)

```
1 touch example_tsr_project/parameters_node.py
2 chmod +x example_tsr_project/parameters_node.py
```

Nowo utworzony węzeł powinien mieć następującą lokalizację:

```
1 ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/example_tsr_project/parameters_node.py
```

5.3 Napisanie kodu dla węzła

Następnie należy napisać nowy węzeł do obsługi parametrów i wysyłania maksymalnej ustawionej prędkości kątowej do robota (zalecane skopiowanie kodu z dołączonych plików lub ich wstawienie, gdyż zachowują wcięcia przy kopiowaniu):

```
1 import rclpy
2 from rclpy.node import Node
3 from rclpy.parameter import Parameter
4 from rclpy.node import ParameterDescriptor
5 from rcl_interfaces.msg import SetParametersResult
6 from geometry_msgs.msg import Twist # Import wiadomości ROS2 typu Twist z pakietu geometry_msgs
7 import time
8
9 class RobotController(Node):
10     """
11     Węzeł odpowiedzialny za wysyłanie maksymalnej prędkości obrotowej wynikającej
12     z ustawionego parametru. Zmiana wartości parametru robot_max_w powoduje zmianę
13     prędkości obrotowej robota.
14     """
15     def __init__(self):
16         super().__init__('robot_controller')
17
18         # Deklarowanie parametrów z wartościami domyślnymi
19         self.declare_parameter('robot_max_v', 0.0)
20         self.declare_parameter('robot_max_w', 1.0)
21         self.declare_parameter('robot_type', 'default_type')
22
23         # Parametr tylko do odczytu
24         self.declare_parameter('robot_id', 'ROBOT_001', descriptor=ParameterDescriptor(read_only=True))
25
26         # Pobranie wartości parametrów
27         self.robot_max_v = self.get_parameter('robot_max_v').value
28         self.robot_max_w = self.get_parameter('robot_max_w').value
29         self.robot_type = self.get_parameter('robot_type').value
30         self.robot_id = self.get_parameter('robot_id').value # Read-only
31
32         # Utworzenie atrybutu klasy będącego Publisherem wiadomości typu Twist na temacie /turtle1/cmd_vel.
33         self.vel_publisher = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
34         # Utworzenie timera, co 0.2s wywołuje się metoda self.timer_callback
35         self.timer = self.create_timer(0.2, self.timer_callback)
36
37         self.get_logger().info(f'Starting robot {self.robot_id} with max speed:\n-linear {self.robot_max_v}
38         ↳ \n-angular: {self.robot_max_w}')
39
40         # Subskrypcja zmiany parametrów w czasie działania
41         self.add_on_set_parameters_callback(self.parameter_callback)
42
43     def parameter_callback(self, params):
44         """
45         Funkcja wywoływana przy zmianie parametrów
46         """
47         # Aktualizacja wartości parametrów
48         for param in params:
```

```

48         if param.name == "robot_max_v" and param.type_ == Parameter.Type.DOUBLE:
49             self.robot_max_v = param.value
50             self.get_logger().info(f'Updated linear speed: {self.robot_max_v}')
51         elif param.name == "robot_max_w" and param.type_ == Parameter.Type.DOUBLE:
52             self.robot_max_w = param.value
53             self.get_logger().info(f'Updated angular speed: {self.robot_max_w}')
54         elif param.name == "robot_type" and param.type_ == Parameter.Type.STRING:
55             self.robot_type = param.value
56             self.get_logger().info(f'Updated type: {self.robot_type}')
57         elif param.name == "robot_id":
58             self.get_logger().warning("Attempt to change read-only parameter 'robot_id' was blocked.")
59
60         # Wyświetlenie informacji o pomyślnym zakończeniu procedury zmiany parametrów
61         return SetParametersResult(successful=True)
62
63     def timer_callback(self):
64         msg = Twist() # Utworzenie wiadomości ROS2 typu Twist
65
66         # Uzupełnienie pól wiadomości
67         msg.linear.x = 2.0
68         msg.angular.z = self.robot_max_w
69
70         # Wyświetlenie wiadomości
71         self.vel_publisher_.publish(msg)
72
73
74 def main(args=None):
75     # Inicjalizacja ROS2 dla programu, musi być wykonana przed utworzeniem jakiegokolwiek węzła (node'a).
76     rclpy.init(args=args)
77
78     # Utworzenie nowego obiektu węzła ROS2 na podstawie klasy RobotController
79     node = RobotController()
80     # Obsługa błędów wynikająca z pojawiania się przerwania z klawiatury ctrl+c
81     try:
82         # Spin umożliwia węzłowi ROS2 działanie i oczekuje na zdarzenia. Brak linii spowoduje natychmiastowe
83         ↪ zakończenie działania programu.
84         rclpy.spin(node) # Obsługa zdarzeń ROS2
85     except KeyboardInterrupt:
86         node.get_logger().info("Shutting down Robot Controller.")
87
88     # Usuwa węzeł przed zamknięciem programu. Zwalnia zasoby pamięci zajmowane przez ten obiekt.
89     node.destroy_node()
90
91     # Zamyka połączenie z ROS2.
92     rclpy.shutdown()
93
94 if __name__ == '__main__':
95     main()

```

5.4 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *entry_points* tak, aby dodać nowy węzeł odpowiedzialny za publikowanie utworzonej przed chwilą wiadomości. W tym miejscu należy dodawać uruchomienie każdego nowego węzła (node).

```

1  entry_points={
2      'console_scripts': [
3          'parameters_node = example_tsr_project.parameters_node:main',
4      ],
5  }

```

Lewa strona w linii z dodanym **parameters_node** odnosi się do unikalnej nazwy w pakiecie, która będzie wykorzystywana podczas uruchamiania nowego węzła. Ta nazwa nie będzie widoczna po uruchomieniu węzła na liście uruchomionych węzłów (`ros2 node list`). Zamiast tego będzie widoczna nazwa wynikająca z konfiguracji węzła w pliku z programem. Wpisana w konstruktorze tam gdzie pojawia się `super().__init__('robot_controller')`. Z tego powodu w zaprezentowanym przykładzie nazwą węzła będzie `robot_controller`.

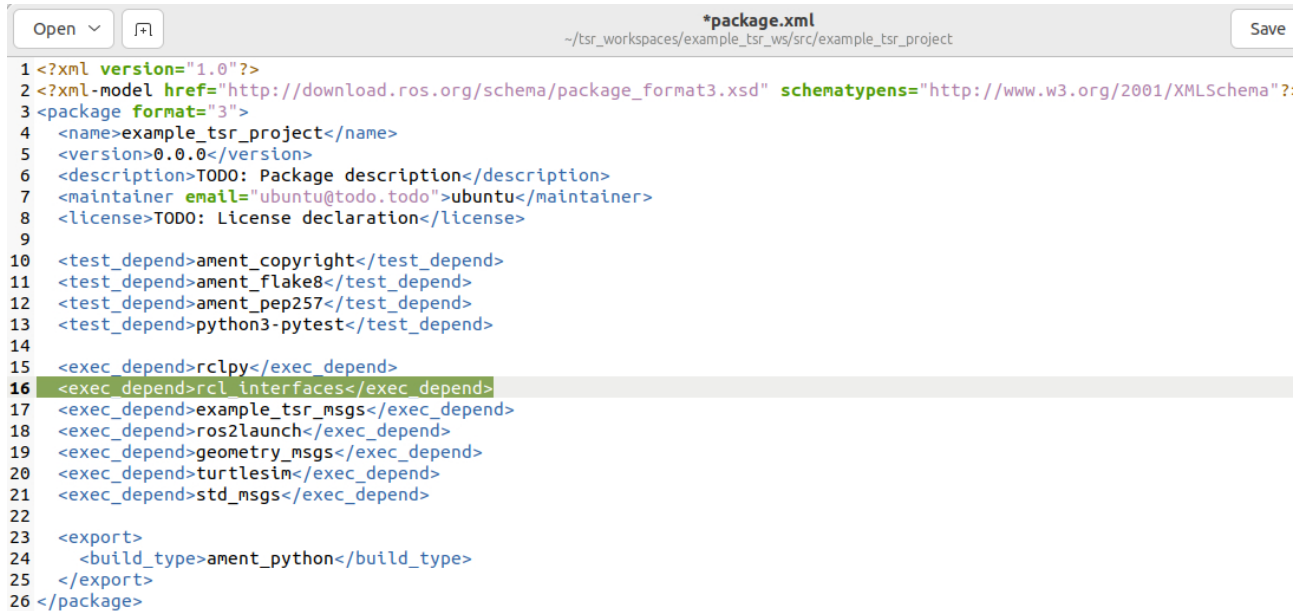
Z prawej strony znajduje się odniesienie do lokalizacji pliku i miejsca w kodzie, z którego uruchamiany jest nowy węzeł. W tym przypadku jest to plik `parameters_node.py` znajdujący się w katalogu `example_tsr_project`. Uruchomienie następuje w `main`.

5.5 Modyfikacja package.xml

Następnie należy dokonać edycji pliku `package.xml` i dodać zależność od obsługi parametrów.

```
1 <exec_depend>rcl_interfaces</exec_depend>
```

Po edycji plik powinien wyglądać następująco (Rys. 5.1):



Rys. 5.1: Plik `package.xml` po edycji dla pakietu `example_tsr_project`

5.6 Budowanie paczki

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczą paczkę (można też zbudować cały workspace):

```
1 colcon build --symlink-install --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

5.7 Weryfikacja działania

W terminalu należy wpisać:

```
1 ros2 run example_tsr_project parameters_node
```

W nowym terminalu można sprawdzić listę dostępnych parametrów (Rys. 5.2):

```

ros2 parubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param list
/robot_controller:
  robot_id
  robot_max_v
  robot_max_w
  robot_type
  use_sim_time
/turtlesim:
  background_b
  background_g
  background_r
  qos_overrides./parameter_events.publisher.depth
  qos_overrides./parameter_events.publisher.durability
  qos_overrides./parameter_events.publisher.history
  qos_overrides./parameter_events.publisher.reliability
  use_sim_time

```

Rys. 5.2: Lista dostępnych parametrów

Parametry dla węzła sterującego prędkościami robota wraz z wartościami (Rys. 5.3):

```

ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param dump /robot_controller
/robot_controller:
  ros__parameters:
    robot_id: ROBOT_001
    robot_max_v: 0.0
    robot_max_w: 5.0
    robot_type: default_type
    use_sim_time: false

```

Rys. 5.3: Lista dostępnych parametrów dla węzła robot_controller

Kolejne testy zmiany wartości parametrów:

```

ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param set /robot_controller robot_max_w 3.0
Set parameter successful
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param set /robot_controller robot_max_w 3
Setting parameter failed: Wrong parameter type, expected 'Type.DOUBLE' got 'Type.INTEGER'
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param set /robot_controller robot_id "134141fse23"
Setting parameter failed: Trying to set a read-only parameter: robot_id.
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 param get /robot_controller robot_max_w
Double value is: 3.0

```

Rys. 5.4: Testy zmiany wartości parametrów

W tym przykładzie wartość parametru `robot_max_w` została zmieniona na 3.0 a w rezultacie otrzymana została odpowiedź o pomyślnym ustawieniu wartości parametru. Należy pamiętać, że typ ustawianej wartości powinien być zgodny z typem parametru. Chyba, że z konfiguracji w węźle wynika inaczej, ale ten przypadek nie będzie omawiany. Próba zmiany tego samego parametru po przekazaniu wartości typu `INTEGER` spowodowała pojawienie się błędu. Nie można również w trakcie działania węzła zmienić parametru `robot_id`, który jest tylko do odczytu.

Poprawność ustawienia nowej wartości dla parametru `robot_max_w` sprawdzona została z wykorzystaniem `get` a w wyniku otrzymano wartość 3.0.

6 Procedura dodania nowych plików konfiguracyjnych do pakietu

Dla utworzonego poprzednio węzła `robot_controller` (uruchomienie `ros2 run example_tsr_project parameters_node`) w pakiecie `example_tsr_project` dodany zostanie nowy plik konfiguracyjny `robot_controller_config.yaml`.

Ponadto zostanie utworzony drugi plik konfiguracyjny zawierający parametry od koloru tła dla węzła od symulacji turtlesim. Procedura zostanie opisana tak, jakby pakiet nie był skonfigurowany do obsługi plików konfiguracyjnych .yaml.

6.1 Przejść do pakietu example_tsr_project

Należy w terminalu przejść do workspace'a (example_tsr_ws) a następnie do src i pakietu example_tsr_project:

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

6.2 Utworzenie podkatalogu config

Utworzenie w pakiecie example_tsr_project katalogu dla parametrów (katalog config. W tym celu należy przejść do lokalizacji głównego katalogu dla pakietu a następnie utworzyć katalog config. Można dodać ręcznie katalog z pominięciem terminala.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
2 mkdir config
```

6.3 Utworzenie plików konfiguracyjnych .yaml

W katalogu config (~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/config) utworzyć plik tekstowy robot_controller_config.yaml. Do pliku należy wkleić przykładową konfigurację dla węzła robot_controller:

```
1 /robot_controller: # parametry konfiguracyjne dla wezła robot_controller uruchamianego z pliku
  ↳ parameters_node.py
2   ros__parameters:
3     robot_id: "SimRobot1"
4     robot_max_v: 1.0
5     robot_max_w: 4.0
6     robot_type: "default"
```

Ponadto należy utworzyć plik do konfiguracji tła (turtlesim_bg.yaml) dla symulacji turtlesim:

```
1 /turtlesim: # parametry konfiguracyjne dla tła symulacji turtlesim
2   ros__parameters:
3     background_b: 0
4     background_g: 255
5     background_r: 0
```

6.4 Aktualizacja setup.py

W pakiecie example_tsr_project w pliku setup.py. Należy zmodyfikować linię z data_files tak, aby dodać linkowanie w trakcie kompilacji pakietu do plików konfiguracyjnych .yaml znajdujących się w katalogu config.

```
1 data_files=[
2     ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
3     ('share/' + package_name, ['package.xml']),
4     (os.path.join('share', package_name, 'config'), glob(os.path.join('config', '*.yaml'))),
5 ],
```

Dokładnie chodzi o linię nr 5, która powinna zostać dodana. Po modyfikacji sekcja data_files z uwzględnieniem plików .yaml może wyglądać następująco


```

data_files=[
    ('share/ament_index/resource_index/packages',
     ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
    (os.path.join('share', package_name, 'launch'), glob(os.path.join('launch', '*.launch.py'))),
    (os.path.join('share', package_name, 'config'), glob(os.path.join('config', '*.yaml'))),
    (os.path.join('share', package_name, 'rviz'), glob(os.path.join('rviz', '*.rviz'))),
],

```

Rys. 6.1: Przykładowa konfiguracja `setup.py` z dodanym linkowaniem plików `.yaml` w trakcie kompilacji.

6.5 Budowanie pakietu

Należy przejść do głównego katalogu dla workspace'a (`example_tsr_ws`):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczy pakiet (można też zbudować cały workspace `colcon build`):

```
1 colcon build --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

6.6 Weryfikacja działania

Z poziomu workspace, w którym nastąpiła kompilacja należy sprawdzić czy pojawił się plik konfiguracyjny we wskazanej lokalizacji. W miejscu `example_tsr_project` dla innego pakietu należy wstawić odpowiednią nazwę.

```
1 ls install/example_tsr_project/share/example_tsr_project/config/
```

Prawidłowy rezultat przedstawia (Rys. 6.2):

```

ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ls install/example_tsr_project/share/example_tsr_project/conf
robot_controller_config.yaml  turtlesim_bg.yaml

```

Rys. 6.2: Wynik wyświetlenia listy plików zainstalowanych po kompilacji.

7 Procedura dodania nowego pliku .launch

W tej części przedstawione zostaną dwa pliki launch.

Pierwszy z nich (`topic_example_launch.py`) będzie odpowiadał za uruchomienie dwóch węzłów: jednego z przykładowym Publisherem (plik `new_publisher.py`) a drugiego z Subscriberem (plik `new_subscriber.py`).

Drugi plik (`multiple_nodes_launch.py`) natomiast będzie zawierał przykład uruchomienia pliku launch (`topic_example_launch.py`), węzła `robot_controller` (plik `parameters_node.py`) wraz z wczytaniem dla niego pliku konfiguracyjnego (plik `robot_controller_config.yaml`)

Następnie zostanie uruchomiony jeden plik (`multiple_nodes_launch.py`).

7.1 Przejść do pakietu `example_tsr_project`

Należy w terminalu przejść do workspace'a (`example_tsr_ws`) a następnie do `src` i pakietu `example_tsr_project`:

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
```

7.2 Utworzenie podkatalogu launch

Utworzenie w pakiecie `example_tsr_project` katalogu dla plików `launch` (katalog `launch`). W tym celu należy przejść do lokalizacji głównego katalogu dla pakietu a następnie utworzyć katalog `launch`. Można dodać ręcznie katalog z pominięciem terminala.

```
1 cd ~/tsr_workspaces/example_tsr_ws/src/example_tsr_project
2 mkdir launch
```

7.3 Utworzenie plików .launch

W katalogu `launch` (`~/tsr_workspaces/example_tsr_ws/src/example_tsr_project/launch`) utworzyć plik tekstowy `topic_example_launch.py`. Do pliku należy wkleić kod odpowiedzialny za uruchomienie węzłów znajdujących się w plikach wykonywalnych `new_publisher.py` i `new_subscriber.py`:

```
1 from launch import LaunchDescription
2 from launch_ros.actions import Node
3
4 # Funkcja zwraca opis LaunchDescription wskazujący na kolejne węzły jakie zostaną uruchomione.
5 def generate_launch_description():
6     return LaunchDescription([
7         Node(
8             package='example_tsr_project', # nazwa pakietu, w którym znajduje się węzeł
9             executable='new_talker_node_name', # nazwa z entry_points w setup.py (po lewo =)
10            name='talker_node', # nazwa węzła, może być taka jak w pliku wykonywalnym albo dowolna (wtedy
11            ↪ można uruchomić ten sam węzeł wiele razy pod inną nazwą.
12            output='screen',
13            parameters=[{'message_frequency': 2.0}] # Przykładowy parametr
14        ),
15        Node(
16            package='example_tsr_project',
17            executable='simple_subscriber',
18            name='subscriber_node',
19            output='screen'
20        )
21    ])
```

Należy również utworzyć drugi plik tekstowy `multiple_nodes_launch.py` odpowiedzialny za uruchomienie plików wykonywalnych: `topic_example_launch.py` i `parameters_node.py` wraz z załadowaniem parametrów z pliku konfiguracyjnego `robot_controller_config.yaml`.

```
1 import os
2 from launch import LaunchDescription
3 from launch.actions import IncludeLaunchDescription
4 from launch_ros.actions import Node
5 from launch.actions import ExecuteProcess
6 from launch.launch_description_sources import PythonLaunchDescriptionSource
7 from ament_index_python.packages import get_package_share_directory
8
9 def generate_launch_description():
10     # nazw pakietu
11     package_name = 'example_tsr_project'
12
13     # Pobranie ścieżki do katalogu pakietu
14     package_dir = get_package_share_directory(package_name)
15
16     # Ścieżka do pliku launch, który uruchamia talkera i subskrybenta
17     topic_launch_file = os.path.join(package_dir, 'launch', 'topic_example_launch.py')
18
19     # Ścieżka do pliku konfiguracyjnego YAML dla węzła parameters_node
20     config_file = os.path.join(package_dir, 'config', 'robot_controller_config.yaml')
21     # Ścieżka do pliku konfiguracyjnego YAML (konfiguracja tła) dla węzła turtlesim
22     bg_color_config_file = os.path.join(package_dir, 'config', 'turtlesim_bg.yaml')
23
24     return LaunchDescription([
25         # Uruchomienie istniejącego launch file
```

```

26     IncludeLaunchDescription(
27         PythonLaunchDescriptionSource(topic_launch_file)
28     ),
29
30     # Uruchomienie węzła parameters_node z załadowaną konfiguracją YAML
31     Node(
32         package=package_name,
33         executable='parameters_node',
34         name='robot_controller',
35         output='screen',
36         parameters=[config_file]
37     ),
38
39     # Wczytanie parametrów koloru tła dla turtlesim - wykonanie polecenia w terminalu ros2 param load
40     ↪ /turtlesim path/turtlesim_bg.yaml
41     ExecuteProcess(
42         cmd=['ros2', 'param', 'load', '/turtlesim', bg_color_config_file],
43         output='screen'
44     )
45 ])
```

7.4 Aktualizacja *setup.py*

W pakiecie *example_tsr_project* w pliku *setup.py*. Należy zmodyfikować linię z *data_files* tak, aby dodać linkowanie w trakcie kompilacji pakietu do plików **launch.py* znajdujących się w katalogu *launch*.

```

1 data_files=[
2     ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
3     ('share/' + package_name, ['package.xml']),
4     (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
5 ],
```

Dokładnie chodzi o linię nr 4, która powinna zostać dodana. Po modyfikacji sekcja *data_files* z uwzględnieniem plików *launch* może wyglądać następująco

```

11 data_files=[
12     ('share/ament_index/resource_index/packages',
13      ['resource/' + package_name]),
14     ('share/' + package_name, ['package.xml']),
15     (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
16     (os.path.join('share', package_name, 'config'), glob('config/*.yaml')),
17     (os.path.join('share', package_name, 'rviz'), glob(os.path.join('config', '*.rviz'))),
18 ],
19 install_requires=['setuptools'].
```

Rys. 7.1: Przykładowa konfiguracja *setup.py* z dodanym linkowaniem plików *launch* w trakcie kompilacji.

7.5 Budowanie pakietu

Należy przejść do głównego katalogu dla workspace'a (*example_tsr_ws*):

```
1 cd ~/tsr_workspaces/example_tsr_ws/src
```

następnie zbudować pojedynczy pakiet (można też zbudować cały workspace `colcon build`):

```
1 colcon build --packages-select example_tsr_project
```

i wykonać ponowny `source` w terminalu (lub uruchomić nowy terminal):

```
1 source install/setup.bash
```

7.6 Weryfikacja działania

7.6.1 Weryfikacja działania pliku topic_example_launch.py

Zakładamy, że wszystkie inne węzły są wyłączone. Na początek uruchomiony zostanie pierwszy plik:

```
1 ros2 launch example_tsr_project topic_example_launch.py
```

Prawidłowy rezultat przedstawia (Rys. 6.2):

```
ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 launch example_tsr_project topic_example_launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2025-03-05-16-26-56-976913-agnieszka-ThinkPad-X1-Extreme-5490
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [new_talker_node_name-1]: process started with pid [5491]
[INFO] [simple_subscriber-2]: process started with pid [5493]
[simple_subscriber-2] [INFO] [1741188418.653261445] [subscriber_node]: Id wiadomości: 123
[simple_subscriber-2] Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[new_talker_node_name-1] [INFO] [1741188418.653434203] [talker_node]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[new_talker_node_name-1] [INFO] [1741188419.607193625] [talker_node]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[simple_subscriber-2] [INFO] [1741188419.608410073] [subscriber_node]: Id wiadomości: 123
[simple_subscriber-2] Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[new_talker_node_name-1] [INFO] [1741188420.607123954] [talker_node]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[simple_subscriber-2] [INFO] [1741188420.608357326] [subscriber_node]: Id wiadomości: 123
[simple_subscriber-2] Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
```

Rys. 7.2: Widok po uruchomieniu pliku launch

Można również sprawdzić listę wyświetlanych tematów (Rys. 7.3) i zobaczyć, że pojawił się na niej /new_unique_topic_name. Nazwa tematu jest zgodna z tematem, na którym nadaje Publisher i nasłuchuje Subscriber.

```
ros2 ubuntu@agnieszka-ThinkPad-X1-Extreme:~/Desktop$ ros2 topic list
/new_unique_topic_name
/parameter_events
/rosout
```

Rys. 7.3: Widok po wyświetleniu listy tematów.

7.6.2 Weryfikacja działania pliku multiple_nodes_launch.py

Należy wyłączyć wszystkie uruchomione węzły. W pierwszej kolejności uruchomimy w jednym terminalu symulację z turtlesim:

```
1 ros2 run turtlesim turtlesim_node
```

Następnie w nowym terminalu uruchamiamy drugi plik launch.

```
1 ros2 launch example_tsr_project multiple_nodes_launch.py
```

Prawidłowy rezultat przedstawia (Rys. 7.4):

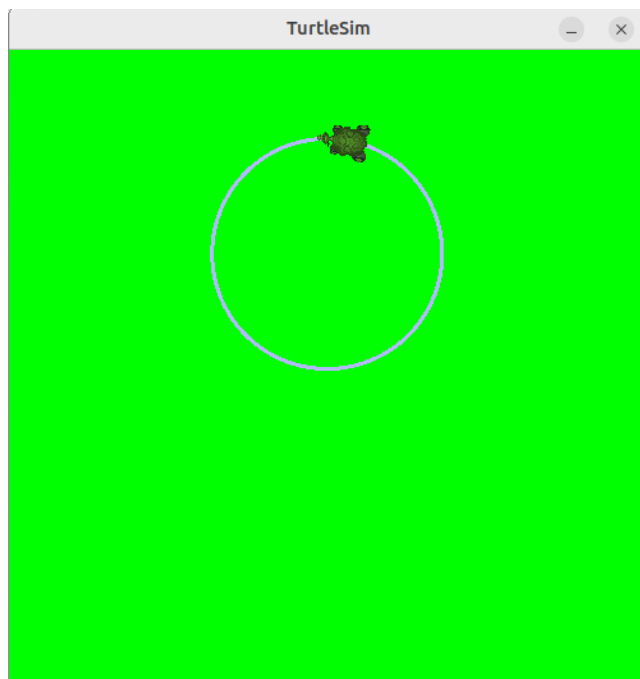
```

ubuntu@agnieszka-ThinkPad-X1-Extreme:~/tsr_workspaces/example_tsr_ws$ ros2 launch example_tsr_project multiple_nodes_launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2025-03-05-17-15-01-728866-agnieszka-ThinkPad-X1-Extreme-9266
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [new_talker_node_name-1]: process started with pid [9267]
[INFO] [simple_subscriber-2]: process started with pid [9269]
[INFO] [parameters_node-3]: process started with pid [9271]
[INFO] [ros2-4]: process started with pid [9273]
[parameters_node-3] [INFO] [1741191302.403388584] [parameters_node]: Starting robot R080T_001 with max speed:
[parameters_node-3] -linear 0.0
[parameters_node-3] -angular 1.0
[ros2-4] Set parameter background_b successful
[ros2-4] Set parameter background_g successful
[ros2-4] Set parameter background_r successful
[INFO] [ros2-4]: process has finished cleanly [pid 9273]
[simple_subscriber-2] [INFO] [1741191303.595626177] [subscriber_node]: Id wiadomości: 123
[simple_subscriber-2] Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[new_talker_node_name-1] [INFO] [1741191303.597328136] [talker_node]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[new_talker_node_name-1] [INFO] [1741191304.553311053] [talker_node]: Opublikowano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))
[simple_subscriber-2] [INFO] [1741191304.554026615] [subscriber_node]: Id wiadomości: 123
[simple_subscriber-2] Odebrano: example_tsr_msgs.msg.ExampleMsgType(id=123, name='Nowa wiadomosc', value=0.0, is_valid=True, dynamic_integer_array=[3, 4, 5], example_goal=geometry_msgs.msg.Point(x=0.0, y=0.0, z=0.0))

```

Rys. 7.4: Widok po uruchomieniu drugiego pliku launch

Ponadto zmienia się kolor tła symulacji turtlesim (Rys. 7.5):



Rys. 7.5: Zmiana tła symulacji turtlesim po uruchomieniu drugiego pliku launch

8 Dokumentacja i inne linki

Rozdział może zawierać dokumentację poleceń, linki do źródeł na podstawie, których powstała instrukcja oraz inne linki pozwalające poszerzyć wiedzę.

- Dokumentacja ROS Humble

<https://docs.ros.org/en/humble/index.html>

- Więcej o parametrach

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Parameters/Understanding-ROS2-Parameters.html>

<https://docs.ros.org/en/humble/Concepts/Basic/About-Parameters.html>

- Więcej o plikach `*launch.py`

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Launch-Main.html>